

Course overview

Introduction to computer programming

Management and Artificial Intelligence



Greetings, folks!

Hello students!

Welcome to the *Introduction to Computer Programming* course!

Web Resources and More

Resources

Web Resources you should have access to

- [LUISS Learn](#)
- [Course Syllabus](#)
- [Rooms and Timing](#)

e-Learning platform

[LUISS Learn](#) is our e-Learning platform.

It is important that you get access!

You'll find there:

- Slides and other teaching material
- Exercises
- Details on exam dates, reservations, results (besides standard Luiss channels)
- Communications from teacher and teaching assistants

Modalità di accesso

- Gli studenti con un account @studenti.luiss.it devono accedere alla piattaforma cliccando in alto a destra sul link **Students Keyless and Luiss accounts Login** e successivamente su **Riconoscimento facciale per studenti**, utilizzando Keyless. Per configurare Keyless, consulta [questa guida](#).
- In caso di problemi con Keyless potete contattare il supporto all'indirizzo <https://support.keyless.io>.
- Per richiedere il reset della vostra password di un account Luiss, potete scrivere a supportoit@luiss.it.
- In caso di diverse problematiche, potete consultare le [FAQ per studenti](#), le [FAQ per docenti](#), la [pagina di tutorial per docenti](#) o, per problemi relativi alle funzionalità della piattaforma, scrivere all'indirizzo elearning@luiss.it.
- Se hai necessità di installare il Respondus Lockdown Browser per eseguire un esame, puoi andare su [questa pagina](#).

To access

- Students with a @studenti.luiss.it account must access the platform by clicking on the **Students Keyless and Luiss accounts Login** link at the top right and then on **Riconoscimento facciale per studenti**. To set up Keyless, see [this guide](#).
- For problems with the keyless authentication, you can log to the site <https://support.keyless.io>.
- To ask a password reset of a Luiss account, you can write to supportoit@luiss.it.
- In case of different problems, you can read the [FAQs for students](#), the [FAQs for teachers](#) or, for problems related to the functions of the platform, write to elearning@luiss.it.
- If you need to install the Respondus Lockdown Browser to take an exam, you can [click on this link](#).

La Luiss mette a disposizione **contenuti didattici in e-learning** volti a supportare la didattica frontale e ad agevolare - in un'ottica di *blended learning* - il lavoro autonomo dello studente necessario ad integrare quanto svolto in classe.

Luiss provides e-learning content to support frontal teaching and facilitating - in a blended learning perspective - the student's independent work to integrate what has been done in the classroom.

Navigation

Home
» Courses

Courses





Dashboard

Customise this page

 Recently accessed courses



Management and computer science
INTRODUCTION TO COMPUTER PRO



Connettiti alle aule virtuali di lezione - Conn
Sale lezioni (da 31 a 40) - Lesson roo



Tutorial per docenti - Tutorials for teachers
Tutorial per docenti sull'uso di Web

After clicking on "My Home"
you'll get a list of the
courses
you are enrolled in.

 Navigation

Dashboard

-  Site home
-  Site pages
-  My courses
 -  Dipartimento di Impresa e Management

 Private files

No files available

[Manage private files...](#)

 Timeline



No upcoming activities due

 Course overview

 All (except removed from view)

 Course name

 Card



Personnel

Teachers

- Teacher: Emilio Coppa
 - Position: Assistant Professor Tenure-Track @ LUISS
 - Research topics: software testing, vulnerability detection
 - Extra: Team Manager of TeamItaly, the italian cybersecurity team
 - Other info: <https://ecoppa.github.io>
 - Office Hours: send me an e-mail so we can arrange a time slot.
 - Contact: `ecoppa@luiss.it`
- Teacher: Alessio Martino
 - Position: Assistant Professor @ LUISS
 - Research topics: machine learning, big data analysis, computational biology
 - Office Hours: send me an e-mail so we can arrange a time slot.
 - Contact: `amartino@luiss.it`

Teaching assistants

- TA: Giorgio Piccardo
 - Office Hours: you can ask info in his first Lab session.
 - Contact: `gpiccardo@luiss.it`
- TA: Fabio Angeletti
 - Office Hours: you can ask info in his first Lab session.
 - Contact: `fangeletti@luiss.it`

Course Info

Course Goals

- The course is intended to teach basic programming concepts to students with no prior coding experience, developing an attitude towards computational thinking.
- It will provide the students with:
 - an understanding of the role that computation can play in solving problems
 - the ability to write small programs to accomplish useful goals

Course Roadmap

The course will familiarise the students with computer programming and will cover the following topics:

- Introduction to computation
- Introduction to programming languages
- Python basics
- Variables, data types, expressions
- Control structures
- Repetition structures
- Input / output
- Functions and modules
- Recursion
- Strings
- Lists
- Dictionaries
- Object-oriented programming and classes

Organization

Lectures + Lab Sessions: *Laptops at your fingertips, always!*

- Monday @ 14:45–16:15
Several rooms in Viale Romania
(often) Lab Session with TA
- Tuesday @ 10:00–11:30
A308 (Viale Romania)
(often) Theory lectures & some exercises
- Wednesday @ 15:45–17:15
A211 (Viale Romania)
(often) Lab Session with TA

Exams

Written exam:

- Coding (70% of the overall grade)
- Multiple-choice Tests (30% of the overall grade)

In light of the LUISS Continuous Assessment verification method, we will be having 2 *in itinere* exams:

- First Intermediate Test (tentative date: 23/10/2024)
- Second Intermediate Test (tentative date: 26/11/2024)

that replace the Coding test and the Multiple-choice test. Students that will not take (or pass) the intermediate tests during the course are required to take the full test at the end of the course.

Exam structure

Each intermediate test is a short exam on the topics covered so far. It will consist of a mixture of coding exercises and theory questions (multiple-choice quiz).

Rules (in brief):

- the sum of the coding parts between the two tests yields 31
- the sum of the theory parts between the two tests yields 31
- a score greater than or equal to 16 in each part is enough for successfully passing the coding + theory tests

Extra bonus points:

- pass the coding + theory test via intermediate tests, you'll get an extra bonus point
- if you pass the overall exam (coding + theory + group project) within the Winter session, you'll get an another bonus point

Computer Programming

Why computer programming?

Most everyday tasks require to perform:

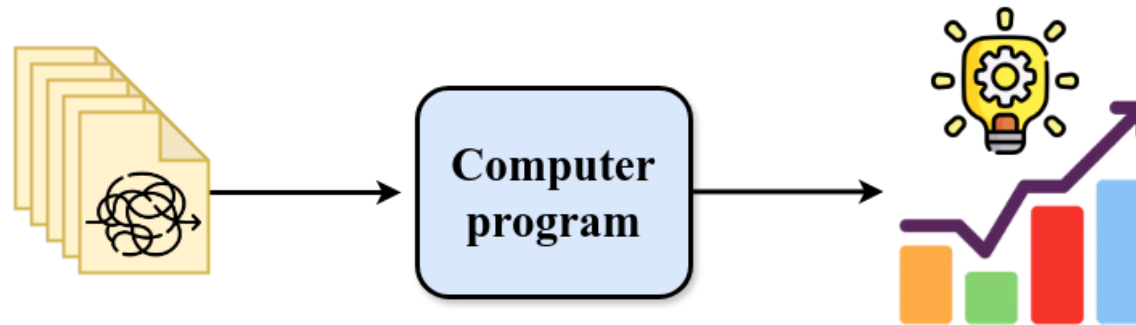
- repetitive...
- long sequences of...
- simple steps

This scenario is *exactly* where a computer excels!

Computers are incredibly fast, accurate, and stupid. Human beings are incredibly slow, inaccurate, and brilliant. Together they are powerful beyond imagination.

-- Albert Einstein

Why do ***you*** need to learn computer programming?



It give us the means to automatically analyze the data and extract knowledge from it.

Solving a task => Algorithm => Recipe followed by a computer

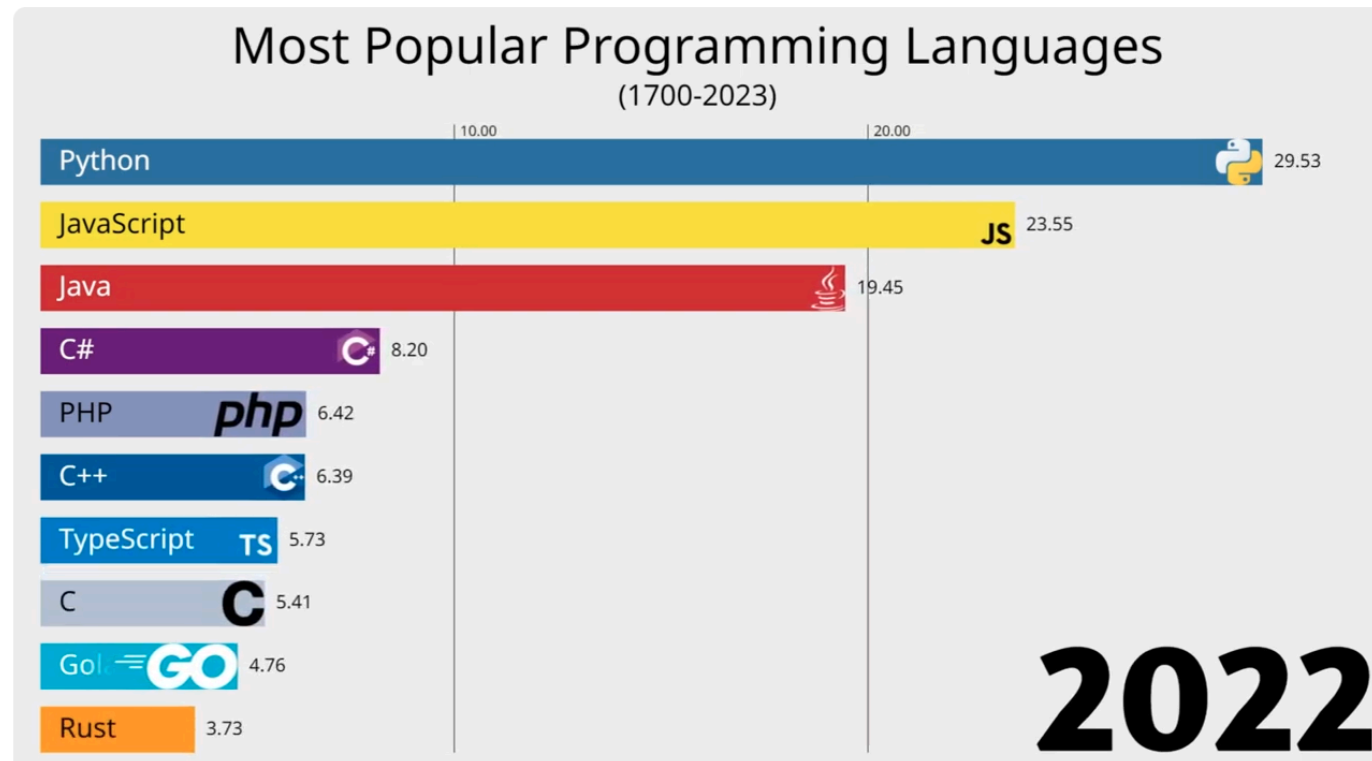
An algorithm features:

- a sequence of simple steps
- flow of control to specify when each step must be executed
- a way of determining when to stop.

...just like a recipe!

However, we cannot write an algorithm in our own natural language (e.g., english). To make an algorithm interpretable by a computer, we need to learn a programming language.

Most Popular Programming Languages



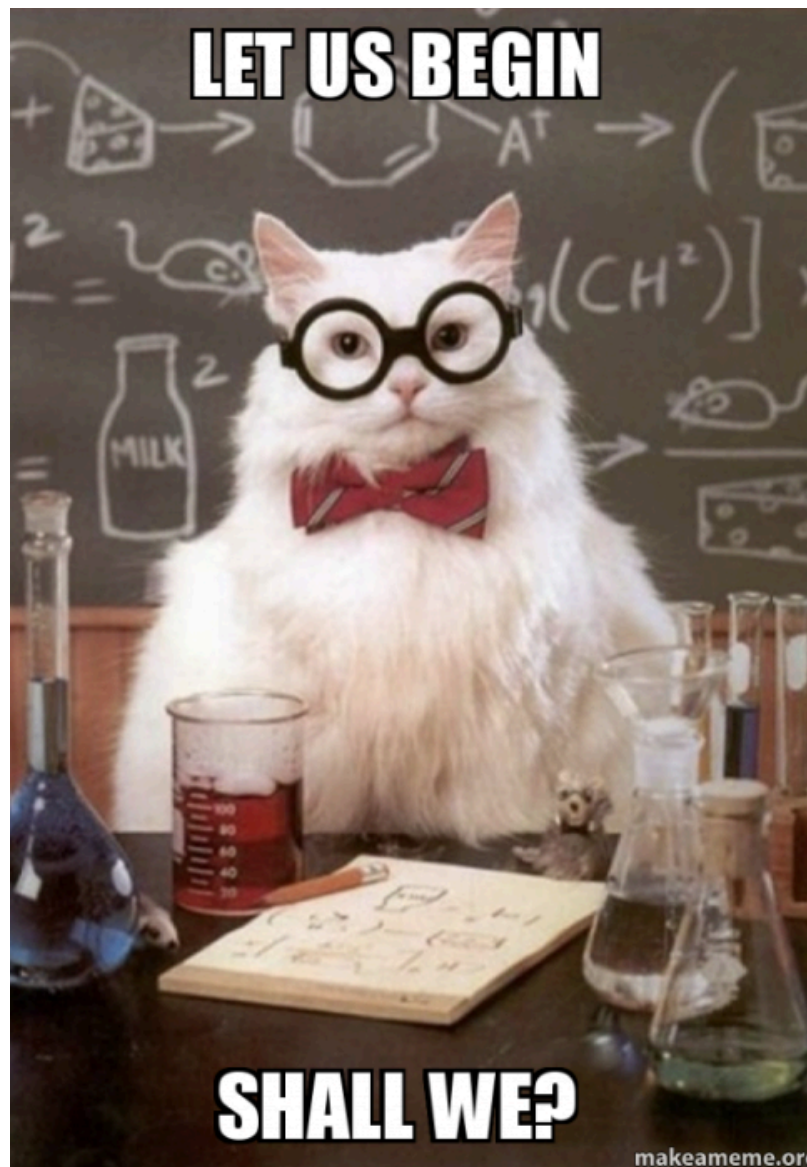
Which Computer Programming Language?



Benefits:

- Easy to learn: designed to be readable, quick to learn
- High-level language: abstracting away most low-level aspects of a computer
- Large community and great support: several libraries allow to build complex projects with just a few lines of code
- Adequate for most tasks: it shows its limits when performance is essential and low-level details play a role. You will hardly work on anything that cannot be implemented in Python

LET US BEGIN



SHALL WE?

makeameme.org

How and Where

How can we run Python code?

Two approaches:

1. Interactive shell:

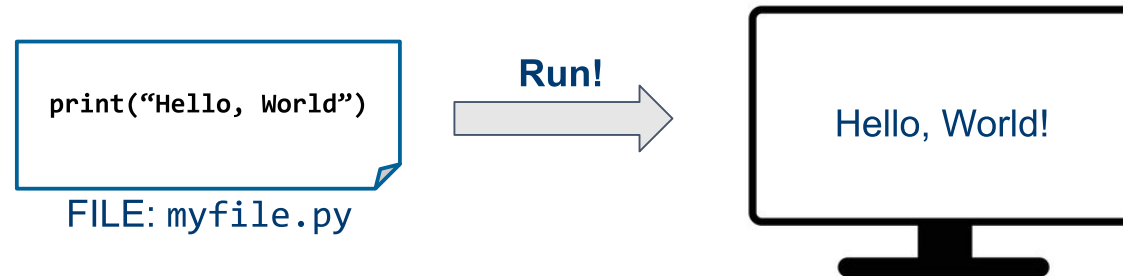
```
Python 3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> █
```

- workflow:
 - start the interactive Python shell
 - write one line of code in the prompt after the symbols `>>>`
 - push enter to execute it and get the results
 - repeat as much as needed
- not very user-friendly
- appropriate only for quick tests
- nothing is saved: you will lose your code when exiting the shell!

How can we run Python code? (cont'd)

Two approaches:

2. Write the code into a textual file and then execute it:



- workflow:
 - write the code into a textual file with extension `.py`
 - execute the file with the Python VM
- This is what you want to do!
- Still *iterative* approach: write the code, check if it works, refine the code, ...

The code can be very cryptic...

Can we add some notes around it?

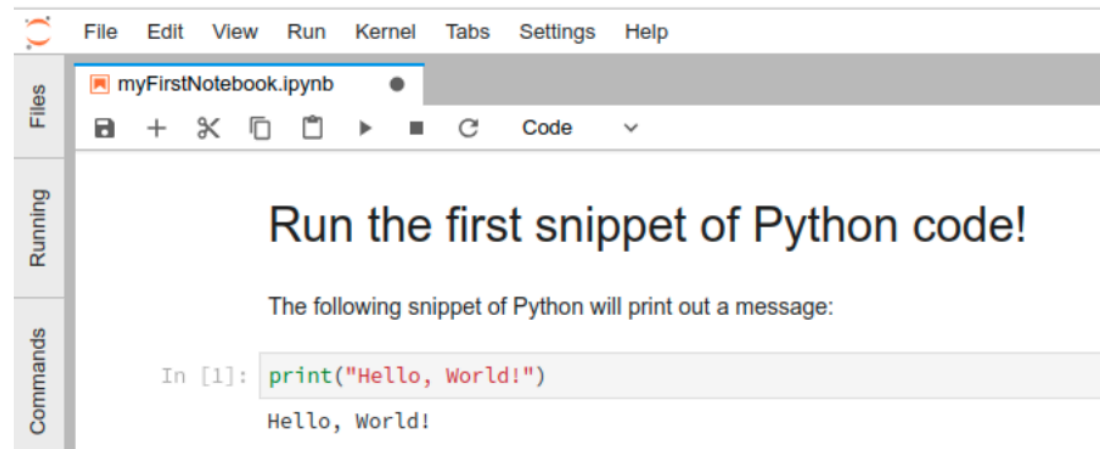
It could be hard to remember why we have written a piece of code, what are its effects and downsides.

It is often convenient to organize our code into "sections" and then add some text to describe what the code is expected to do. This is called:



Jupyter Notebook

Using the environment JupyterLab we can create a new notebook:



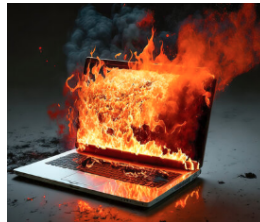
Text section
describing what we
is relevant for us
(humans!)

snippet of code

result of the code
(after running it with
the play button)

Where do we run our code/notebooks?

1. Local setup: your computer!



- you have to install Python and all the other bits
- limited by your computational power

2. Cloud setup:



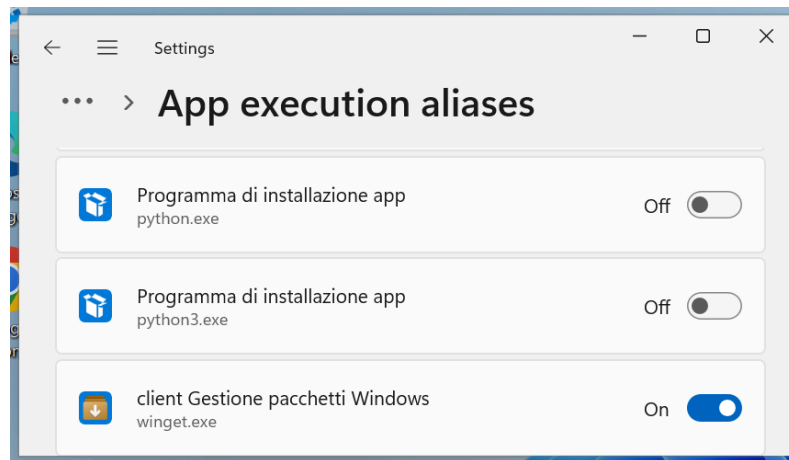
- no need to install everything
- limited by your money

Local Setup

[Only on Windows 11]

Disable Windows Store Aliases

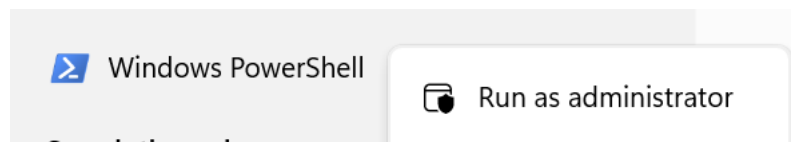
1. From the start menu, search and then open *Manage app execution aliases* (*Alias di esecuzione delle app*)
2. Disables aliases for *python.exe* and *python3.exe*:



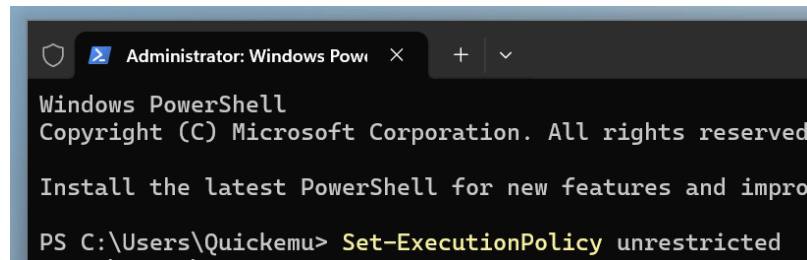
[Only on Windows 11]

Enable script cmdlet

1. From the start menu, search *Windows Powershell*, then using the right-click select *Run as administrator*.

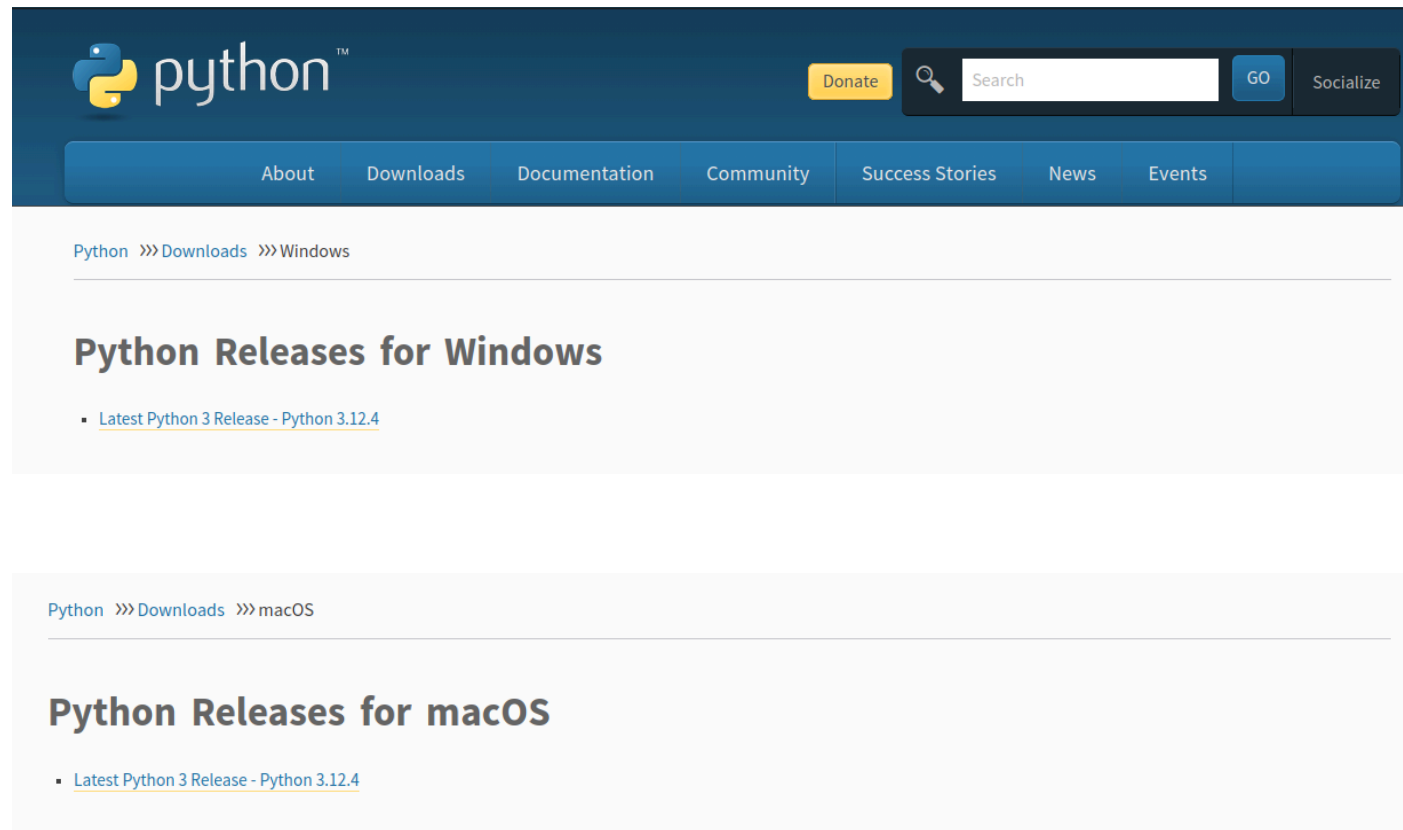


2. Type `Set-ExecutionPolicy unrestricted`, then run it and reply y (or "s" if your Windows is in italian) :



Download Python

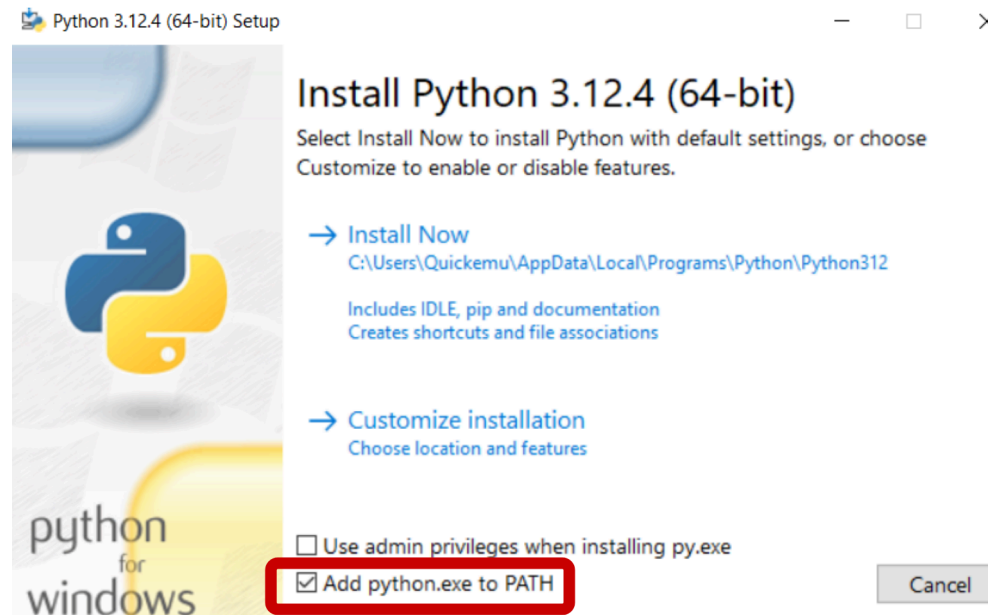
Visit: <https://www.python.org/>



Launch the installer

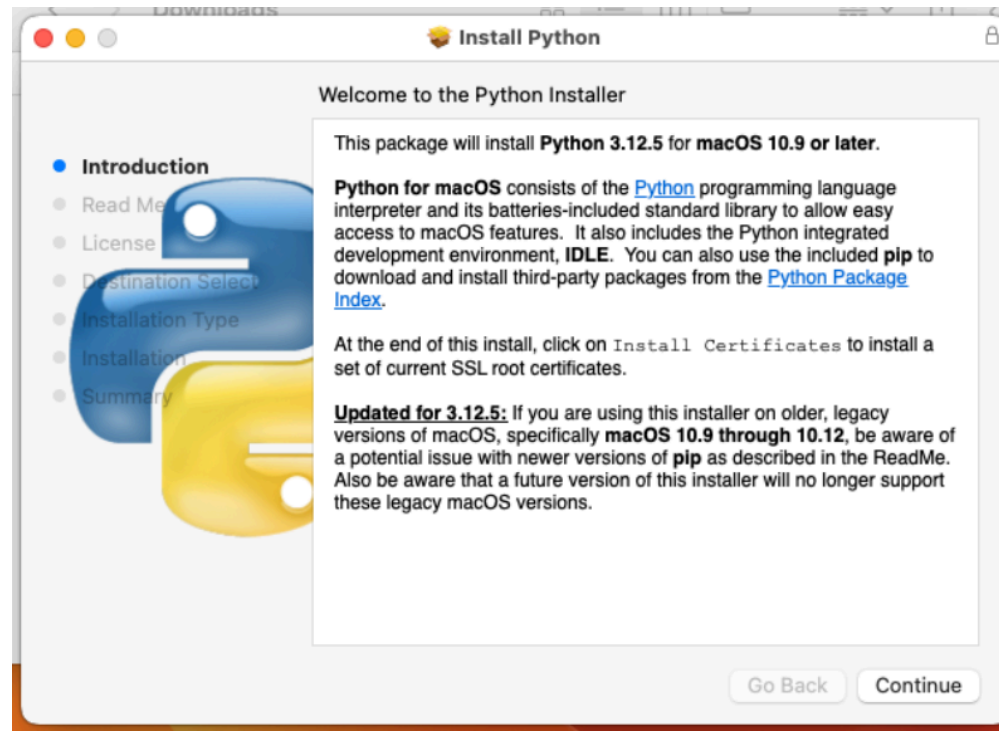
Double click on the downloaded file, e.g., on Windows:

1. Enable ***Add python.exe to PATH***
2. Click *Install now*



Launch the installer (cont'd)

Similarly on Mac OS X:



Install a (good) code editor: **Visual Studio Code**

1. Visit: <https://code.visualstudio.com/Download>
2. Download the latest version for your OS

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.


↓ Windows
Windows 10, 11

User Installer x64 Arm64
System Installer x64 Arm64
.zip x64 Arm64
CLI x64 Arm64

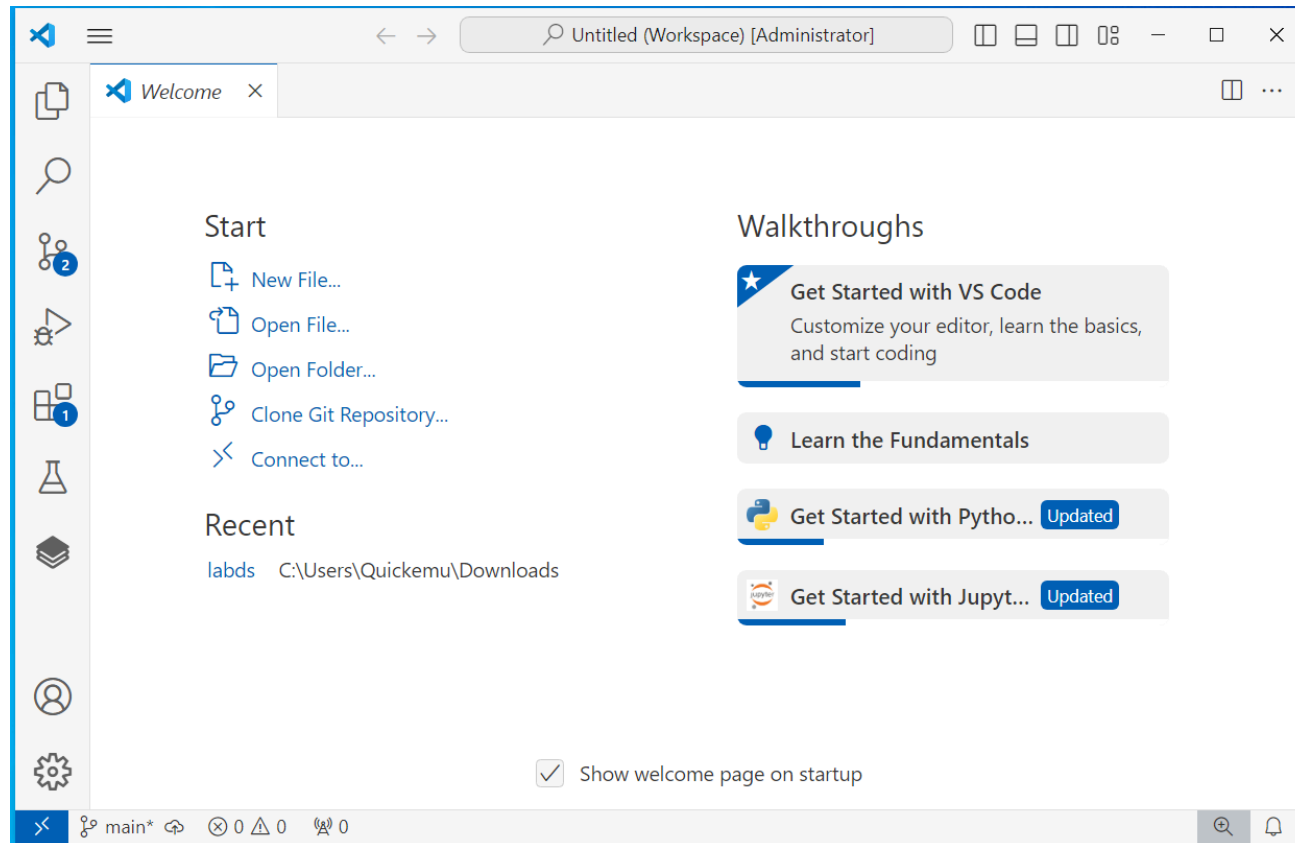

↓ .deb Debian, Ubuntu **↓ .rpm** Red Hat, Fedora, SUSE

.deb x64 Arm32 Arm64
.rpm x64 Arm32 Arm64
.tar.gz x64 Arm32 Arm64
Snap Snap Store
CLI x64 Arm32 Arm64

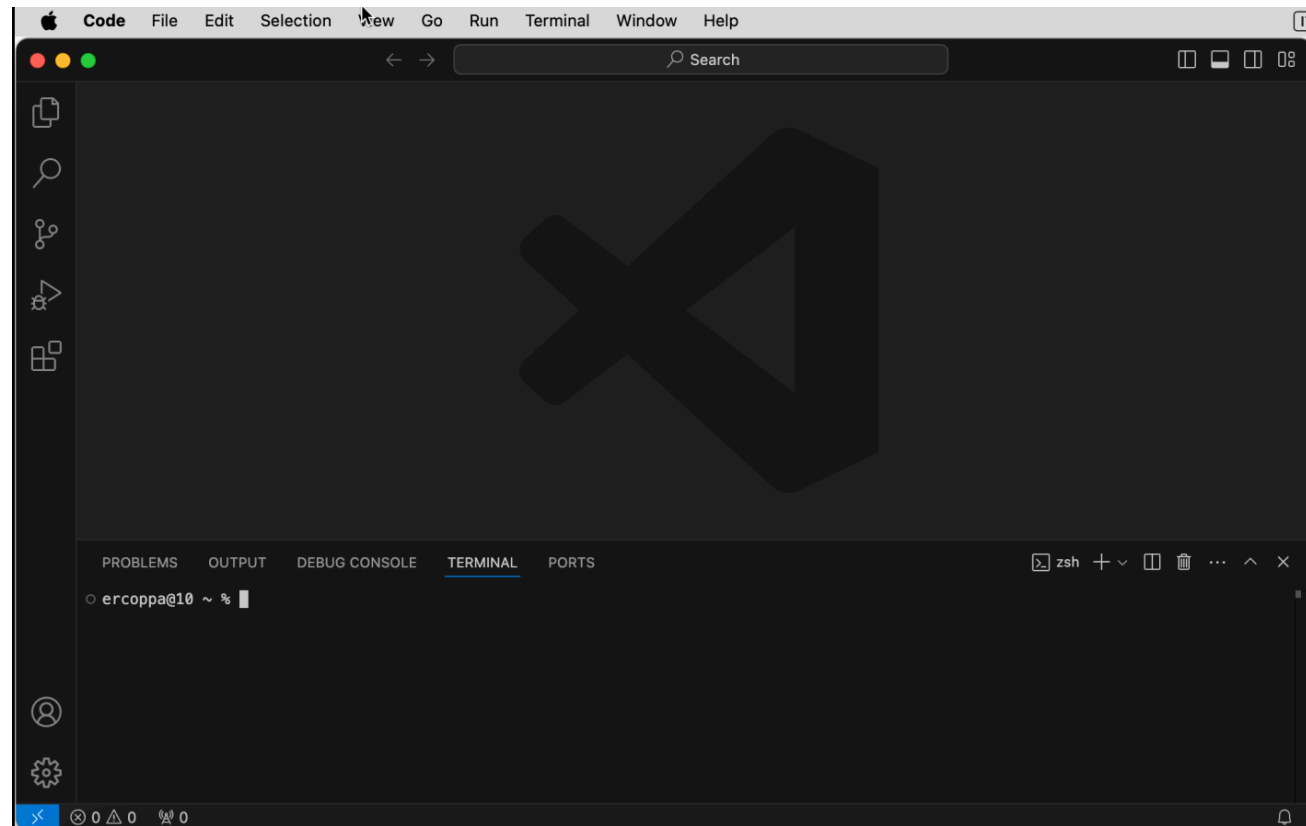

↓ Mac
macOS 10.15+

.zip Intel chip Apple silicon Universal
CLI Intel chip Apple silicon

Start Visual Studio Code (Windows)

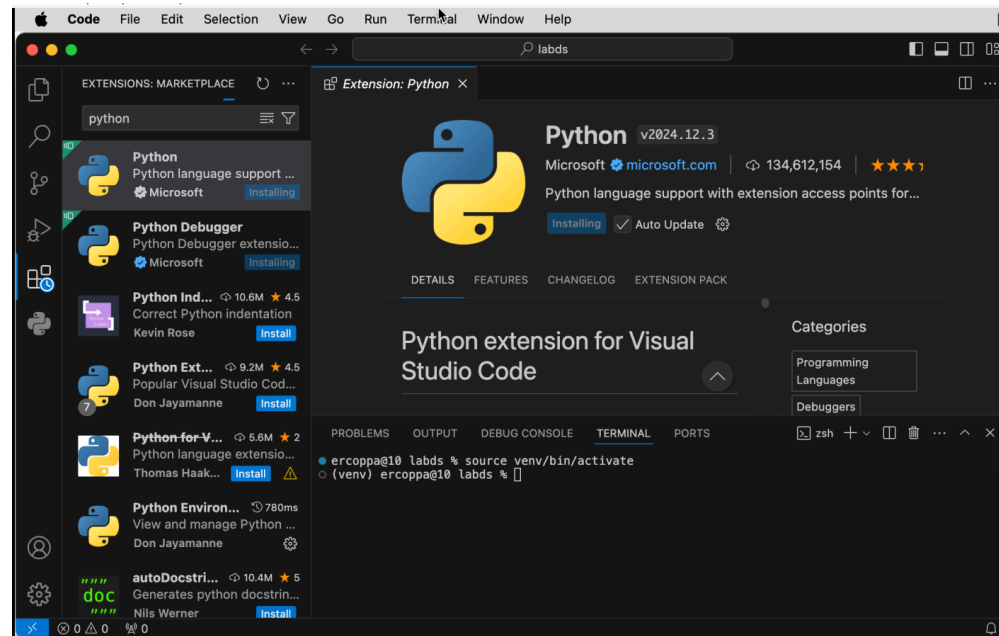


Start **Visual Studio Code** (Mac OS X)



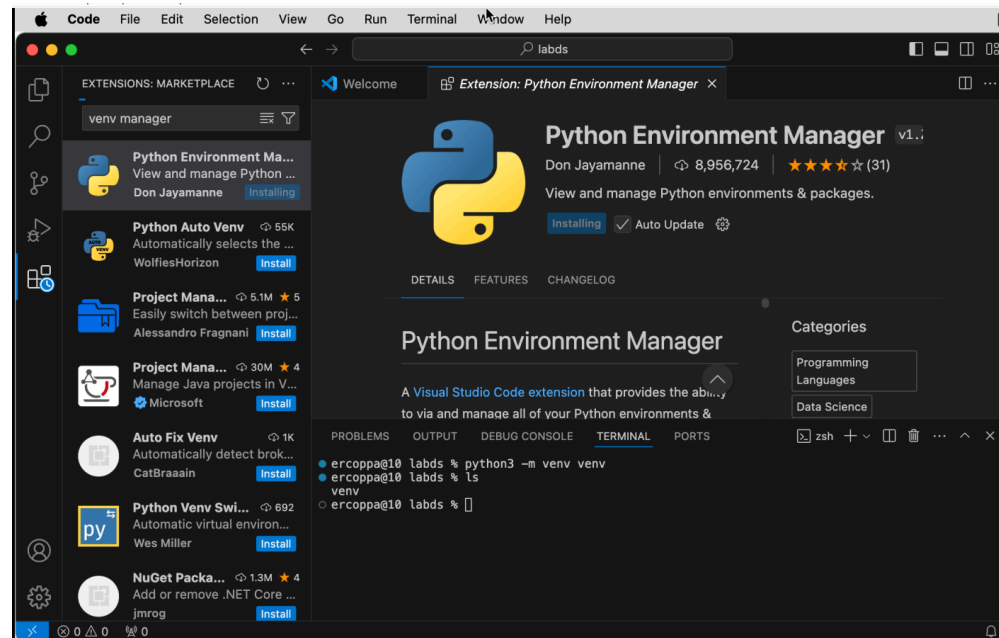
VSCode extension: Python integration

1. From the side bar, open the *Extension* tab
2. Search for *Python*
3. Install the extension



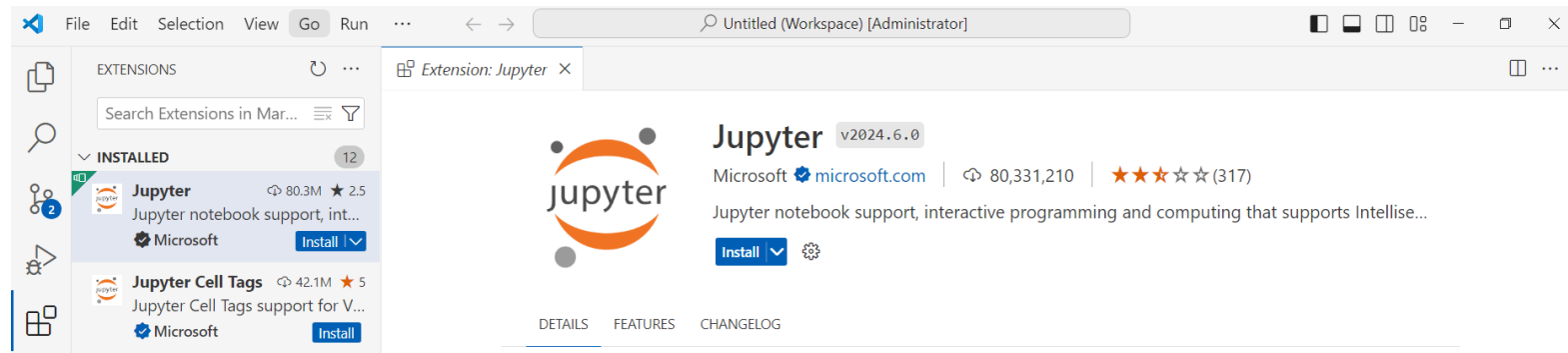
VSCode extension: Python Environment Manager

1. From the side bar, open the *Extension* tab
2. Search for *Python Environment Manager*
3. Install the extension



VSCode extension: Jupyter

1. From the side bar, open the *Extension* tab
2. Search for *Jupyter*
3. Install the extension



Where to install Python packages from the community

We can install Python packages from the community following two strategies:

- *System-wide*: this may create conflicts between different projects using Python.
 - pros: you need to install a package only once for all projects
 - cons: one project may break due to a package installed/updated for another project

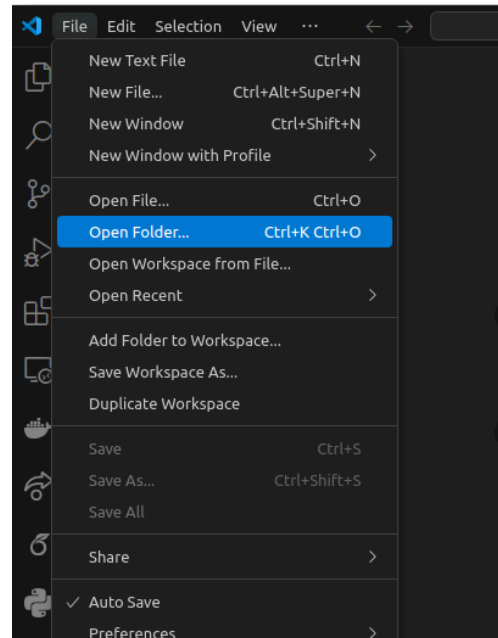
It is now the deprecated approach.

- *Project-wide*: each project has its own *Virtual Environment*.
 - pros: no conflicts across packages of different projects
 - cons: a package must be installed again for each project

This is nowadays the suggested approach and we will follow it.

Create a new virtual environment

1. Create a new folder (e.g., `labds` within `Desktop` folder)
2. Open the folder from VSCode:



3. Start the terminal: *View > Terminal*

4. Type in the terminal:

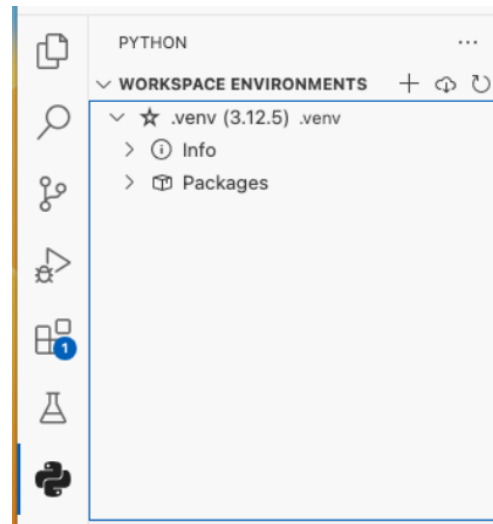
```
python -m venv .venv
```

or:

```
python3 -m venv .venv
```

Pick the default virtual environment

1. From the side bar, open the Python logo
2. Click on the *star* to select the virtual environment



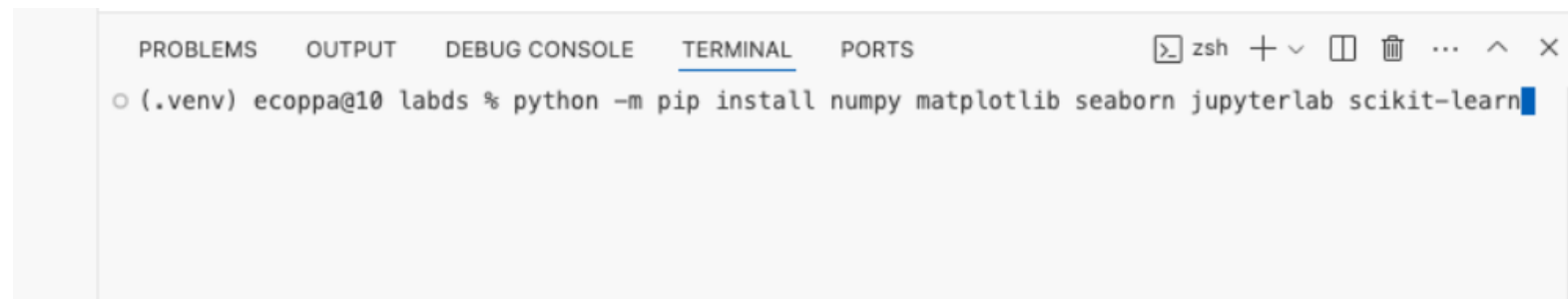
3. Close VSCode and then re-open your project folder in VSCode

Installation of Python packages in the virtual environment

We will use some extra Python bits from the community, including: `numpy`, `matplotlib`, `seaborn`, `scikit-learn`, and `jupyterlab`.

From VSCode:

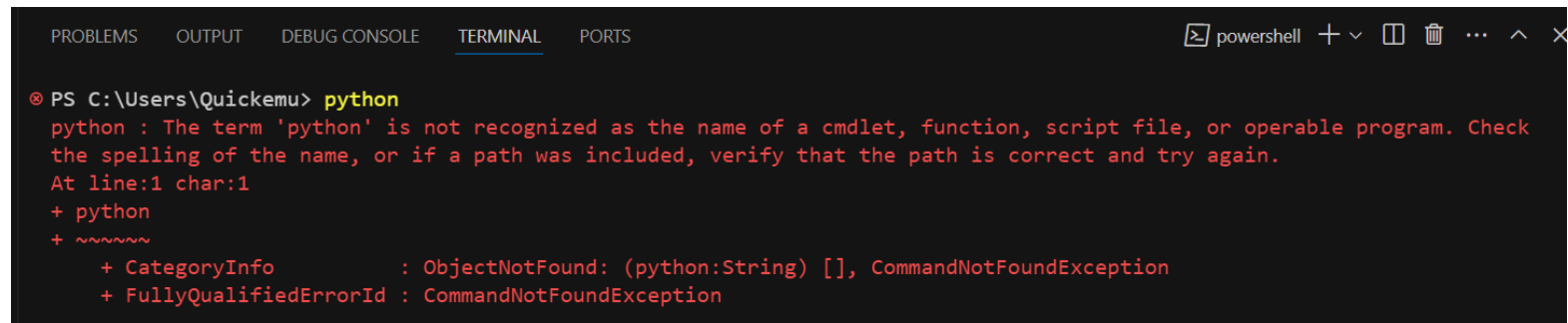
1. Start a new terminal using the menu: *Terminal > New Terminal*
2. `python -m pip install numpy matplotlib seaborn scikit-learn jupyterlab`



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal prompt is `(.venv) ecoppa@10 labds %`. The command `python -m pip install numpy matplotlib seaborn jupyterlab scikit-learn` has been entered and is currently being typed, with a blue cursor at the end of the line. The terminal window includes standard VS Code interface elements like tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, and PORTS, and a toolbar with icons for terminal operations.

Troubleshooting on Windows: python is not recognized

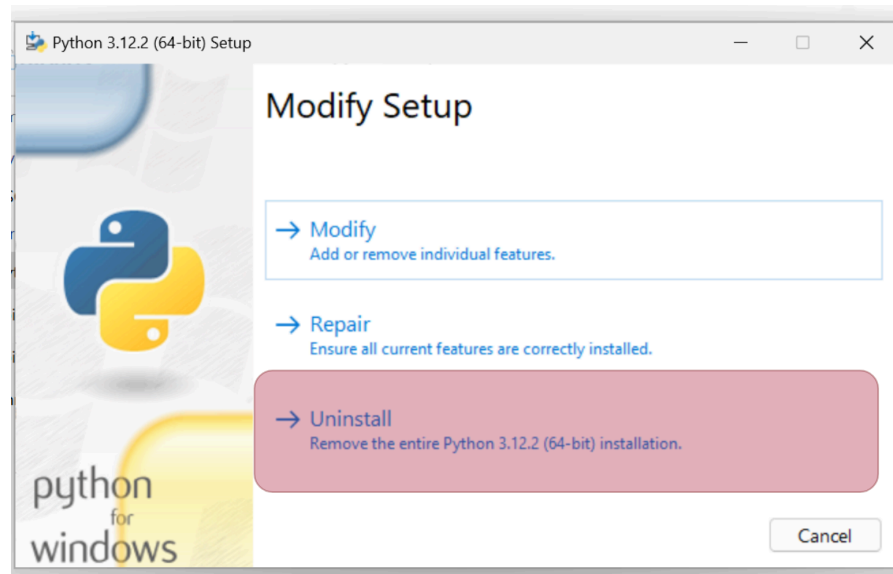
If, when when running `python` in the terminal of VSCode, you get the following error:

A screenshot of a PowerShell terminal window within the Visual Studio Code interface. The terminal shows a command prompt at 'C:\Users\Quickemu>' where the user has entered 'python'. The output is a red error message: 'python : The term 'python' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.' Below this, it says 'At line:1 char:1' followed by a list of error details: '+ python', '+ ~~~~~', '+ CategoryInfo : ObjectNotFound: (python:String) [], CommandNotFoundException', and '+ FullyQualifiedErrorId : CommandNotFoundException'. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The title bar of the terminal window says 'powershell'.

Then, you have to:

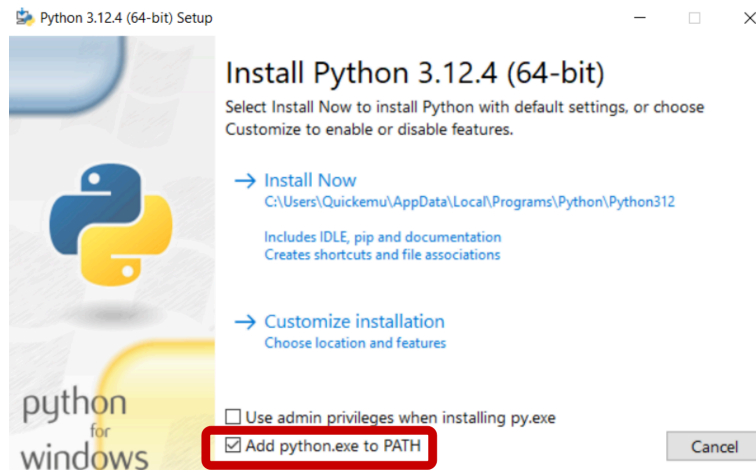
Troubleshooting on Windows: python is not recognized (cont'd)

1. Uninstall Python: run again the installer of Python and select *Uninstall*:



Troubleshooting on Windows: python is not recognized (cont'd)

2. Start the Python installer
3. Select *Add to path* at the bottom



4. Proceed with *Install now*

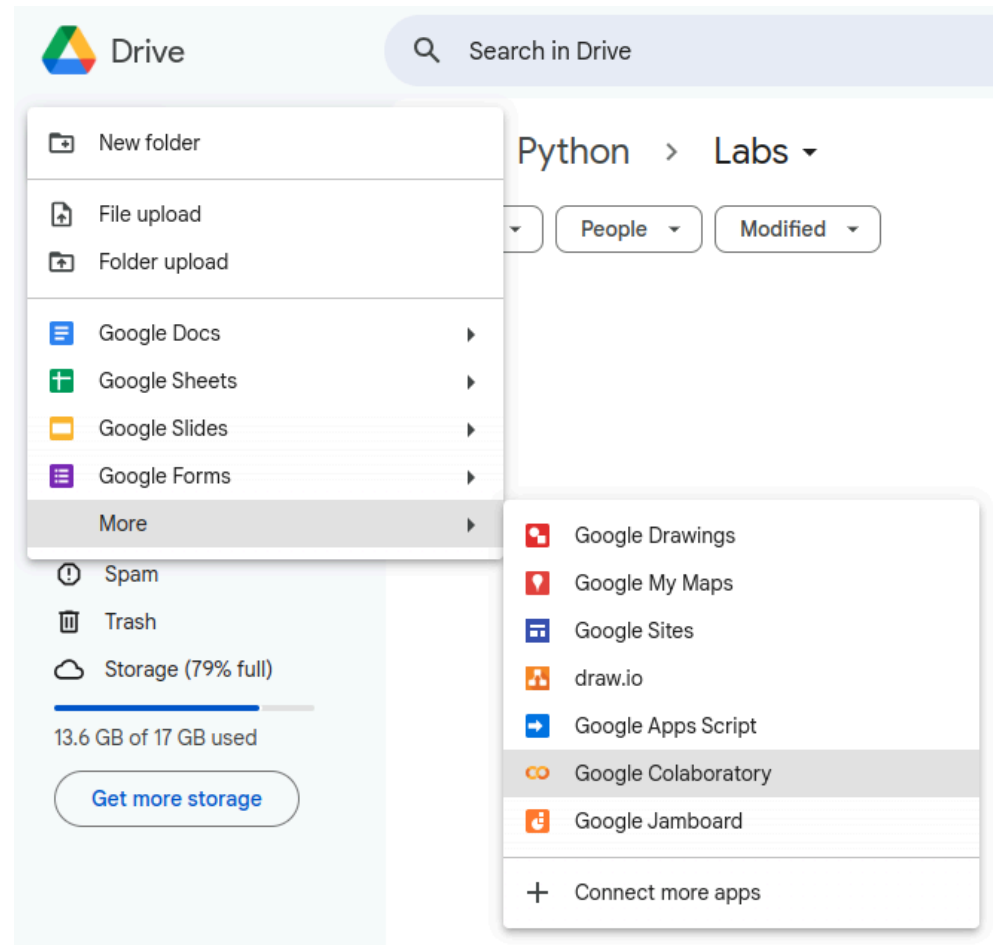
Cloud Setup

Google Colab, i.e., the lazy approach



- Platform from Google
- Google Account is required
- Free of charge to use for the basic plan
- Write code in the browser (no extra is needed)
- Code executed on Google cloud servers
- Low/medium performance, privacy issues
- No need to install anything

From Google Drive



>

Environments

Environment: local setup with VSCode

Use VSCode:

- Quickly write and execute python files (`.py`)
- Quick access to the local terminal
- Write and execute notebooks (`.ipynb`). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Good editor with several extensions (integrations with third-party services)
- Everything runs on your machine

This is the environment that we suggest you to use.

Environment: local setup with JupyterLab

Use JupyterLab:

- Start it from any terminal (e.g., from VSCode) with:

```
python -m jupyterlab
```
- Write and execute notebooks (`.ipynb`) with support even for advanced and weird features.
- Basic editor
- Everything runs on your machine

We suggest to use JupyterLab only when you need to run advanced Jupyter notebooks obtained from the external world

Environment: cloud setup with Colab

Use Google Colab:

- You can work from any computer. Even your tablet (although, it is a bit inconvenient!)
- Write and execute on the cloud notebooks (.pynb). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Basic editor
- Everything must be saved on Google Drive
- Not adequate in future courses where you need a bit of computational power (assuming you do not want to pay Google).

We suggest to use Colab only if you issues with your local setup

Course Material

What you need

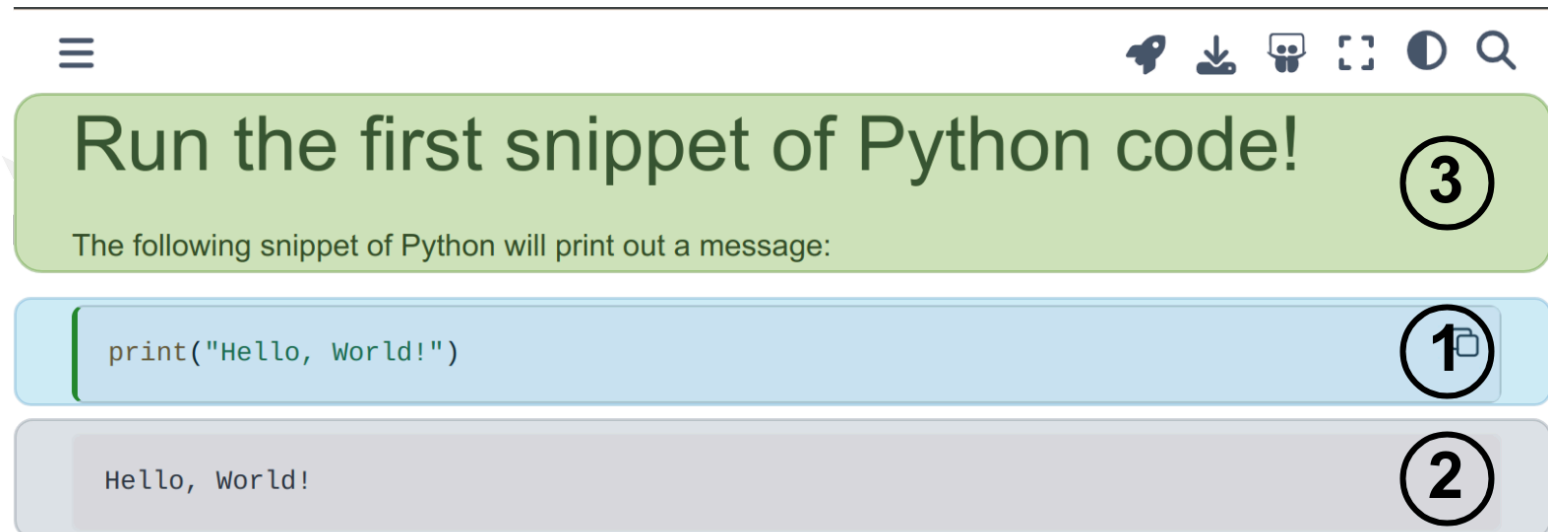
Scan the qrcode to get everything:



<https://ercoppa.github.io/labds>

Each page is a *Notebook*

For instance: <https://ercoppa.github.io/labds/00/00-myFirstNotebook>



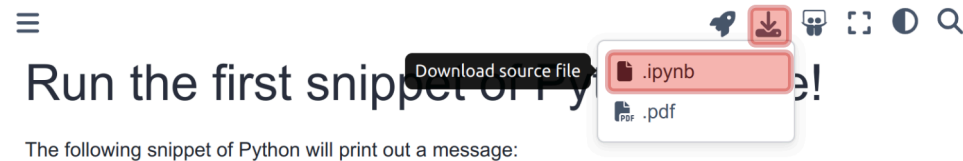
A *notebook* mixes:

1. *code*: something that we can run
2. *result of the code*: we can observe the output of the code execution
3. *text*: used to describe what the code does

You can replay and extend my notebooks!

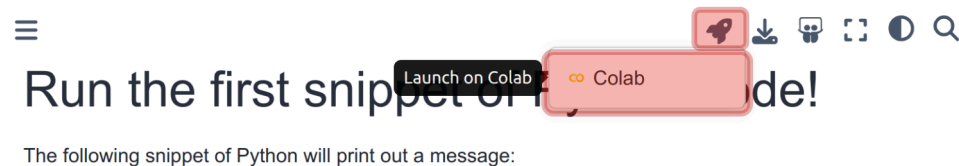
Depending on your setup:

1. Local setup. Run each notebook on your machine. Download the `.ipynb` is easy from the website:



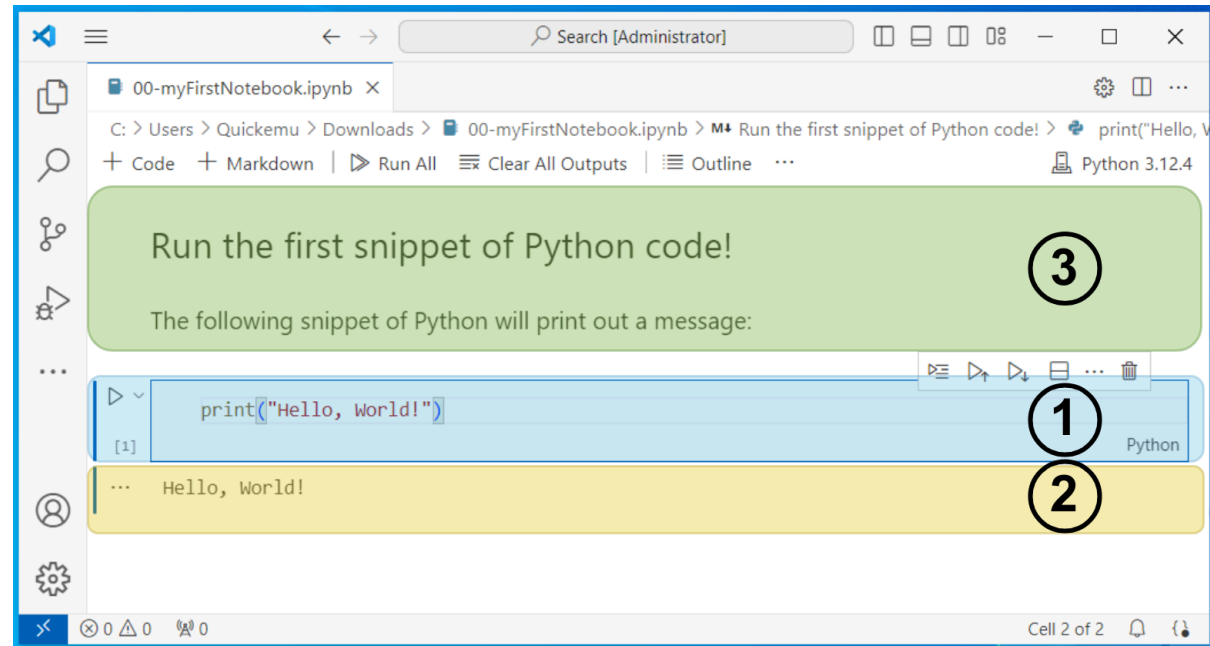
then open it with VSCode or JupyterLab.

2. Cloud setup. Run each notebook on Google Colab (i.e., Google servers). Everything within the browser. To run it on Google Colab:



When opening a notebook on VSCode

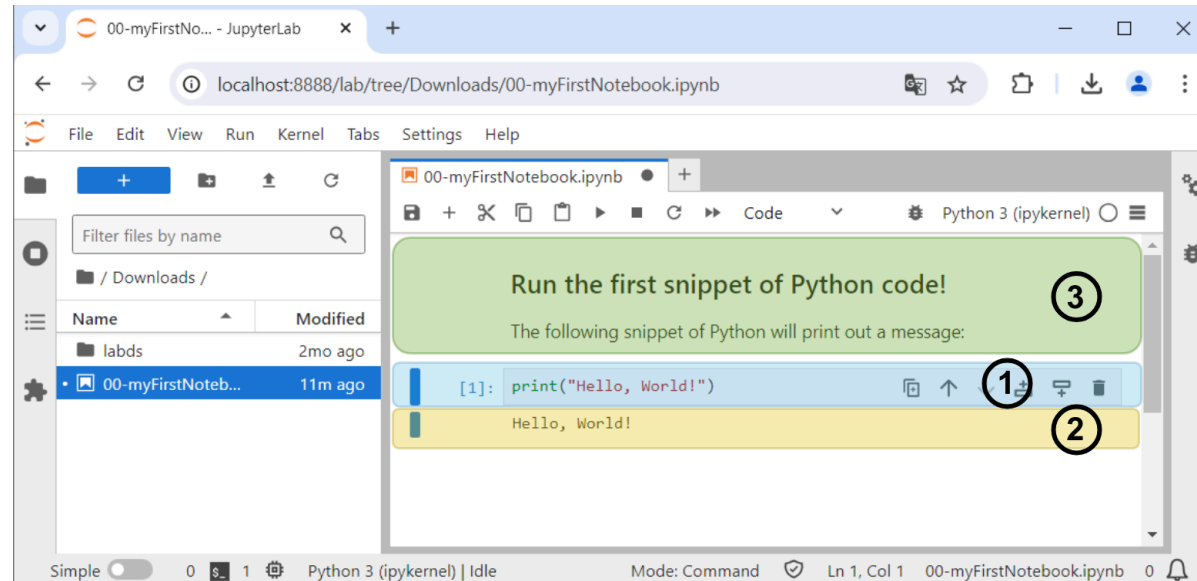
We get the same structure as before:



However, now you can interact with the notebook in several ways.

When opening a notebook on JupyterLab

We get the same structure as before:



However, now you can interact with the notebook in several ways.

When opening a notebook on Colab

We get the same structure as before:

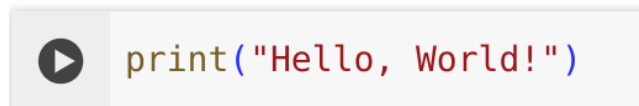


However, now you can interact with the notebook in several ways.

Replaying or updating a notebook

Regardless of the environment (VSCode, JupyterLab, or Colab), we can perform similar interactions with the notebook.

For instance, on Google Colab, if we select a code *cell*, then we get a play button:

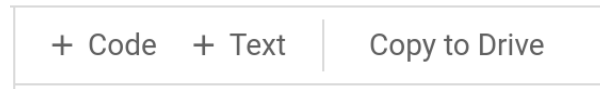


If we push it, we can run the code and (re)generate the execution output:



Moreover, we can modify the notebook:

- double click on existing cells: change their content
- add new code or text cells:



Now, what?

Now, we need to learn a bit of Python!

Check out the page/notebook *Python Basics*.

